Mitochondria Segmentation Using Geometric Flow-Based Methods

McGill University COMP400 Thesis
Bridget Liu
yitong.liu@mail.mcgill.ca

I. Introduction

The level-set methods are efficient computational techniques for modelling interface motion across various fields [1]. The basic idea is to define a signed distance function embedding ϕ for the source image I, where the zero level-set of ϕ represents a curve $C = \{(x, y)|\phi(x, y) = 0\}$, and the sign of the level-set function represents the inside and the outside regions [2], as shown in Fig. 1. By updating ϕ in time, the curve C can be evolved in normal direction based on:

$$\frac{\partial \phi}{\partial t} = \frac{\phi^{n+1} - \phi^n}{\Delta t} = F\|\nabla \phi\|$$

Depending on how speed (energy) term F is defined, different flows can be obtained. A simple example is the mean curvature flow, where $F = \kappa$. The flow has the property that it evolves such that the curve length is deceased as efficiently as possible [3].

Michailovich et al. proposed a methodology where F is calculated based on the Bhattacharyya coefficient: $B(p_1, p_2) = \int_z \sqrt{p_1(z)p_2(z)dz}$. By generalizing the framework of majority of image segmentation methods, this gradient flow maximizes the Bhattacharyya distance between density estimates of the inside and the outside regions [4].

The goal of this paper is to employ the Michailovich flow to experiment with the segmentation for mitochondria in astrocytes. As the key organelles for energy supplying and metabolic substrates generating, mitochondria tend to cluster in certain astrocytic compartments during neuropil-associated processes [5]. Thus, by analyzing the distribution of mitochondria within astrocytes, we will be able to understand their roles and functions. These findings have further implications for understanding astrocyte and mitochondrial dysfunction, which has been linked to various CNS disorders [6].

The problem is also interesting as there are no automatic or semi-automatic segmentation pipelines in place for extracting mitochondria from focused ion bean scanning electron microscopy (FIB-SEM) images. The ground truth segmentation of our data is obtained by segmenting by hand, the work is rather a laborious task and does not scale well. In this paper, we apply a geometric flow to the data instead, based on the assumption that the intensity distribution inside and outside the mitochondria is different in astrocyte body.

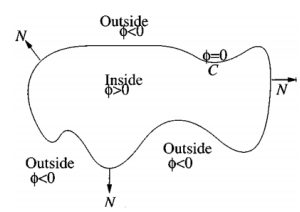

Fig. 1: Curve C propagating in normal direction [7]

A. Overview

We will start by introducing the active contours driven by Michailovich flow in Section II. Then in Section III, we will discuss the implementation details, including data pre-processing, contour-initialzing and parameter tuning. Further, we will extend the model to 3D in section IV, where approaches attempted to speed up computational time is also covered. We will then conclude by presenting the results in both two and three dimensions and suggest future works that can be done.

II. Method

A. Bhattacharyya Flow

The flow is started by transforming image $I(x) \in \Omega$ into a vector based feature image $\boldsymbol{J}(x) \in \mathbb{R}^N$, where N is the number of features. Also, the Heaviside function $\mathcal{H}$ is defined as:

$$\mathcal{H}(\tau) := \begin{cases} 1, & \text{if } \tau \geq 0 \\ 0, & \text{if } \tau < 0 \end{cases}$$

Given a level-set function $\phi(x)$, following kernel-based estimates of the probability density distributions

on the inside and the outside can then be computed respectively as:

$$\begin{cases} P_-(z|\phi) = \dfrac{\int_\Omega K_-(z - \boldsymbol{J}(x))\mathcal{H}(-\phi(x))dx}{\int_\Omega \mathcal{H}(-\phi(x))dx} \\ P_+(z|\phi) = \dfrac{\int_\Omega K_+(z - \boldsymbol{J}(x))\mathcal{H}(\phi(x))dx}{\int_\Omega \mathcal{H}(\phi(x))dx} \end{cases}$$

where $z \in \mathbb{R}^N$, and $K_-(z)$ and $K_+(z)$ are two scalar functions that satisfy $\int_{\mathbb{R}^N} K_-(z)dz = \int_{\mathbb{R}^N} K_+(z)dz = 1$.

Most segmentation methods seek to minimize mutual information shared between the object (inside) and the background (outside) to obtain optimal contour. Gerneralizing this idea, Michailovich flow maximizes the Bhattacharyya distance $\hat{B}(\phi(x))$ between the inside's and outside's pdfs. After differentiation and introduction of a time parameter t, the gradient flow of $\phi(x)$ that minimizes distance is:

$$\phi_t(x) = -\frac{\delta \hat{B}(\phi(x))}{\delta \phi(x)} = -\delta(\phi(x))V(x) \qquad (*)$$

where $\delta(\cdot)$ is the delta function,

$$\begin{aligned} V(x) = &\frac{1}{2}\hat{B}(\phi(x))(A_-^{-1} - A_+^{-1}) \\ &+\frac{1}{2}\int_{z\in\mathbb{R}^N} K_+(z - \boldsymbol{J}(x))\frac{1}{A_+}\sqrt{\frac{P_-(z|(\phi(x)))}{P_+(z|(\phi(x))}}dz \\ &-\frac{1}{2}\int_{z\in\mathbb{R}^N} K_-(z - \boldsymbol{J}(x))\frac{1}{A_-}\sqrt{\frac{P_-(z|(\phi(x)))}{P_+(z|(\phi(x))}}dz \end{aligned}$$

, A_- and A_+ are the areas of inside and outside regions, and $K(z)$ is the kernel used for computing estimated densities.

The flow $(*)$ can be too sensitive to measurement noises and errors in the data. Thus, [4] introduced a regularization term to constrain the length of the active contour. The gradient flow related to solving the optimization problem is now given by:

$$\phi_t(x) = \delta(\phi(x))(\alpha\kappa - V(x))$$

Finally, to solve numerical issues, note that the support of the level-set evolution can be altered by replacing $\delta(\cdot)$ by a smoother $\delta_\Delta(\cdot)$ with finite support and user-controlled Δ [4]. For example, we could define:

$$\delta_\Delta(\phi(x)) = \begin{cases} \frac{1}{2\Delta}\left(1 + \cos\left(\frac{\pi\phi(x)}{\Delta}\right)\right), & \text{if } |\phi(x)| \le \Delta \\ 1 - (\frac{x}{\delta})^2, & \text{otherwise} \end{cases}$$

B. 2D implementation

As Gaspard demonstrated in his implementation for 2D images with 2 channels [8], following aspects were taken into account when it comes to updating the empirical distributions.

1) Kernel-based estimation: The discrete form of kernel estimates is more appropriate to the practical case, where we have pixels in source images. It was shown in his work that compared to Gaussian kernel, parabolic Epanechnikov kernel gives a more robust result, where

$$h_\delta(x) = \begin{cases} 0, & \text{if } x \in [-\delta, \delta] \\ 1 - (\frac{x}{\delta})^2, & \text{otherwise} \end{cases}$$

2) Defining the bandwidth: In [4], the level-set function was only updated on support $\phi^{-1}([-\Delta, \Delta])$ in each iteration. In [8], densities were computed in two regions:

- Inside: $\{x \in \Omega : \phi(x) \in ([-\infty, 0])\}$
- Narrow bands: $\{x \in \Omega : \exists y \in \phi^{-1}(0) \text{ s.t. } x - y < \eta\}$

By minimizing the size of outside's density we are estimating, computational time was able to be speeded up dramatically. We will come back to this in detail in Section III(C).

C. 3D adaption

In level-set form, the 3D version of gradient flow is the same. We need now an input feature matrix of size $\boldsymbol{J}(x) \in \mathbb{R}^{d\times m\times n} \times \mathbb{R}^N$, where (d, m, n) represents the depth, rows, and columns of the source image. Provided that the input data is grayscale, the number of features N is in fact 1.

Also, we generalized the isophote curvature term in 2D defined as

$$\frac{2d_x d_y d_{xy} - d_{yy}d_x^2 - d_{xx}d_y^2}{(d_x^2 + d_{yy}^2 + \epsilon)^{3/2}}$$

to

$$\frac{2(d_x d_y d_{xy} + d_x d_z d_{xz} + d_y d_z d_{xz}) - B}{(d_{xx}^2 + d_{yy}^2 + d_{zz}^2 + \epsilon)^{3/2}}$$

where $B = (d_{yy}+d_{zz})d_x^2 + (d_{xx}+d_{zz})d_y^2 + (d_{xx}+d_{yy})d_z^2$, and ϵ is added to avoid zero denominator [2][9].

III. Experiment on 2D image

A. Data

We were given 7 sets of image data, each set includes a raw image of approximately $400\ pixels \times 400\ pixels$ and a boolean astrocyte mask. As the goal is to segment mitochondria that exists inside astrocyte, we started by cropping based on the mask. An example is shown in Fig. 2.

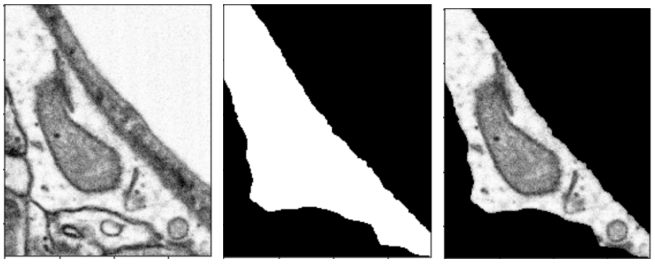

Fig. 2: Original raw1, its boolean astrocyte mask, and the image after cropping

B. Initialization

Then, initial contour is needed for the flow to evolve. Two methods were implemented.

1) Local-max initialization: Peaks in an image can be found by using the skimage.feature.peak_local_max package. Using these peaks as centers, we define a function *local_max(img, thres, r)* to generate circles, where *img* is the source image, *thres* is the minimum distance between any two peaks, and r is a list of radii.

Fig.3 presents two contours generated from raw image 1 with *local_max(raw1,10,5)* and *local_max(raw1,17,8)* respectively.

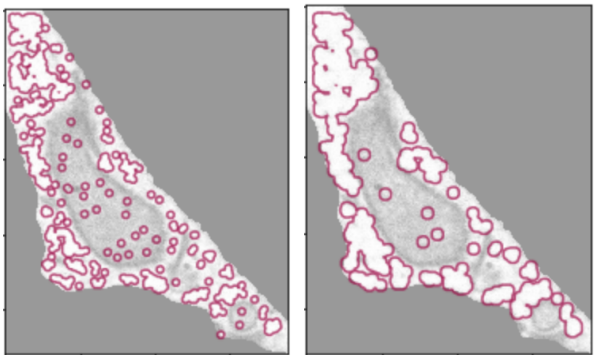

Fig. 3: Local-max initialization on raw1

2) Random dense initialization: Instead of centering at peaks, we initialized circles centering at completely random positions. We defined a function *dense(img,n,r)*, where a total number of *n* centers are selected. We also tried to select one point out of each $k \times k$ grid on source image as center compared to completely randomized centers. It turned out later that gradient flow only takes longer to run, but significantly improved.

The contours generated by *dense(raw1,150,8)* and *dense(raw1,50,15)* are shown in Fig. 4.

C. Parameters

Δ: As the support of $\delta_\Delta(\cdot)$, it affects the size of the area that we need to estimate. In general, smaller Δ leads to a longer computational time and a more stable result. For our 2D data, $\Delta = [5, 10, 15]$ is tested and 5 generally yields the best result.

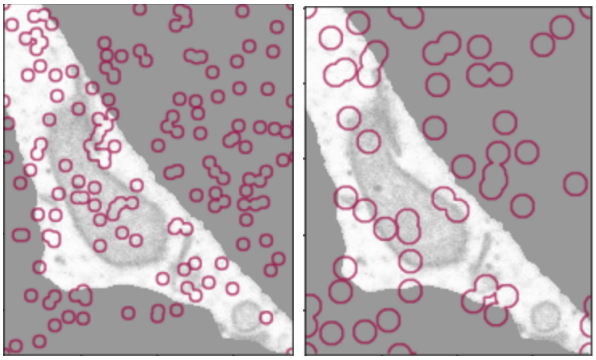

Fig. 4: Random dense initialization on raw1

η: If the narrow band threshold is too small, then there is a chance that no samples will be picked up. In [8]'s implementation, the number of samples in the narrow band in each iteration is computed to avoid this situation. However, if the band is too wide, it will take too long to compute. Here, out of a range of $\eta = [5, 7, 9]$, 5 appears to be optimal.

n_{iter}: The number of iterations needed. Here we set $n_{iter} = 150$, a larger number than the actual iterations needed, for all cases. This is because the flow will collapse at a point when no samples are picked up in the narrow band, and automatically stop.

$level_set_reset_period$: We need to reset the level set to be signed distance from the contours once in a while. Setting it to 10 would help to obtain a clear interior boundary.

α: Since $\alpha\kappa$ is the regularization parameter for dirac approximation, α stands for the importance of curvature relatively to the energy derivative. In theory, larger the α, smoother the boundary. We end up using $\alpha = 10^{-5}$ to balance the details that need to be pick up and the noises in the image.

D. Result

After carefully choosing parameters, following results were obtained by running the Michailovich flow on raw1 and raw7.

For raw1, contour is initialized by *dense*. The process of boundary collapsing as iteration proceeds is shown in Fig. 5. Whereas for raw7, we used *local-max* to set up initial contour. Fig. 6 shows how the contour evolves throughout the 100 iterations. [1]

[1]Animated contour evolving for all experiments available at https://github.com/bridget72/mito/tree/master/anim

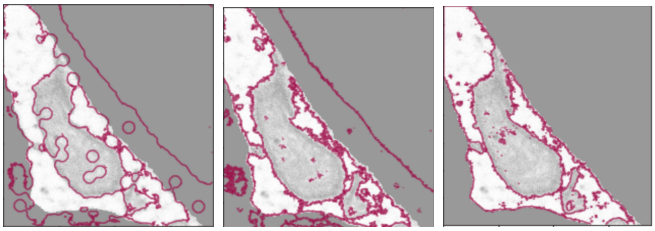

Fig. 5: Flow on raw1 at $n_{iter} = [10, 50, 80]$

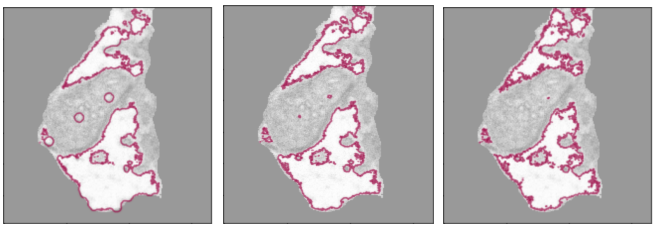

Fig. 6: Flow on raw7 at $n_{iter} = [20, 40, 100]$

Then, a post-processing step was taken to smooth the boundary. We took the final level set ϕ obtained above and run the mean curvature flow for a few steps, where we define [9]:

$$\frac{\partial \phi}{\partial t} = \nabla \cdot (\frac{\nabla \phi}{\|\nabla \phi\|})\|\nabla \phi\|$$

This motion corresponds to a geometric version of the heat equation [2], which can be used to relax and reshape boundaries.

In Fig. 7, we show the contour obtained from Michailovich et. al flow and the smoothed after post-processing. Fig. 8 shows the ground truth segmentation of mitochondria on the left, and the mitochondria mask generated from the smoothed final ϕ. We then calculated the Sorensen-Dice coefficient, where $DSC = 2TP/(2TP + FP + FN)$ to be 0.762.

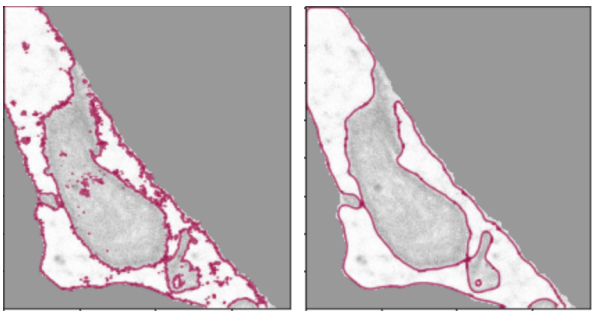

Fig. 7: final phi vs. smoothed contour on raw1

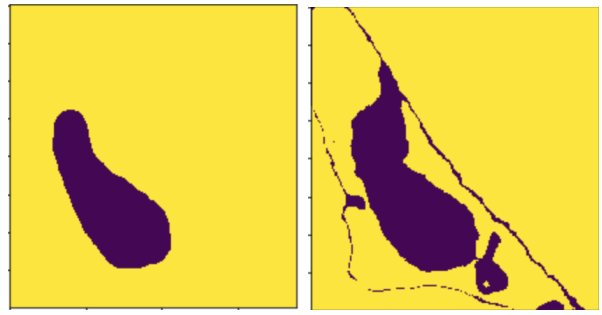

Fig. 8: Boolean mask from flow result vs. ground thruth segmentation on raw1

IV. Experiment on 3D image

A. Data

Now we have a raw stack of size $(179, 666, 890)$ and its astrocyte mask. In a similar fashion as 2D, masked stack is generated. Since the size of image data is too large, running Michailovich flow on the whole stack would take too long to complete. To save time, the stack is cropped in a sense that maximum structure of the mitochondria will be preserved.

B. Initialization

Instead of setting up circles on a slice, we generalized the functions in III(B) to initialize spheres on a stack. It appears that local-max initialization leads to better result. In Fig. 7, we show the contour obtained on a stack of size $(20, 200, 200)$, from both random dense (left) and local-max (right) initialization with exact same parameters. Specifically, we plotted the slices at depth 10 and at iteration 40, where a total of 130 iterations were ran. It can be concluded that the gradient flow initialized by random dense circles takes longer for each iteration and leads to a rather noisy boundary.

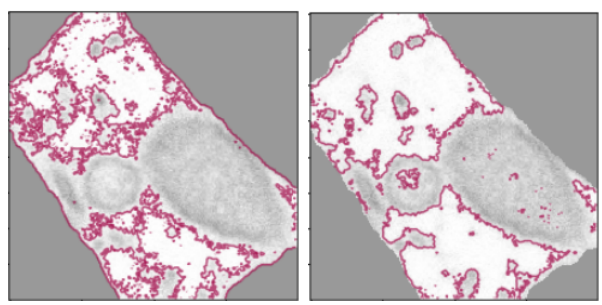

Fig. 9: Flow with different initialization in 3D

C. Result

A complete mitochondria can be found on a stack of size $(179, 320, 290)$, on which it takes approximately

55 hrs and nearly 130 iterations for the gradient flow to segment boundary. To speed up, we cropped the mitochondria half in depth to size $(100, 320, 290)$, such that the whole structure is still kept on each slice.

In the following plots, we show the small spheres generated initially with *local-max*, and the region's boundary after running the gradient flow for 110 iterations. In Fig. 8, both contours are overlaid on a 2D slice that captured at $depth = 90$. In Fig. 9 and Fig. 10, we rotated the stack clock-wise by 180 degrees around z-axis, purely for presenting a clearer result. Only the region boundaries, i.e., where $\phi(x) = 0$, is plotted in red.

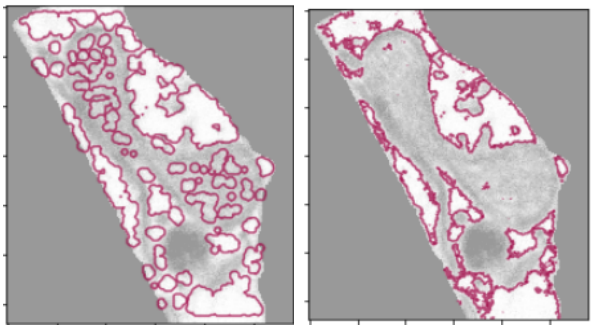

Fig. 10: Initial vs. final contour in 2D

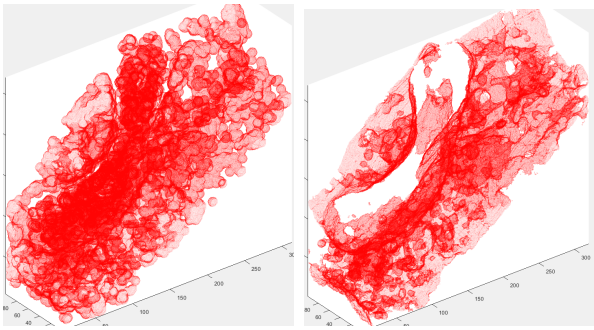

Fig. 11: Initial vs. final contour in 3D

Using the same smoothing algorithm as mentioned in III.D and changing only dimension, the post-processed contour was obtained as shown in Fig. 12. The boolean mask generated by the smoothed contour and the ground truth segmentation drawn by hand are shown in Fig. 13. We plot the results on two slices of the stack, at depth 50 and depth 90 respectively. We then calculated the dice coefficient same as before, taking the two boolean masks as input. The score was calculated to be 0.594, which makes sense since the model detects wherever intensity varied. In Fig. 13, we can see that not only mitochondria, but other organelles such as ER are considered interior during segmentation. Also, the close neighbour of the boundary of the astrocyte was also picked up due to the fact we pre-processed the data with astrocyte mask.

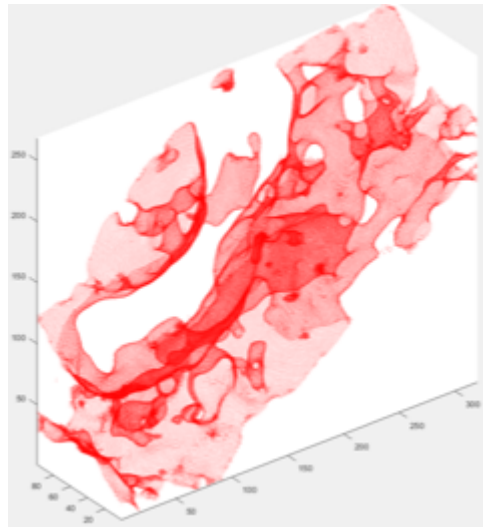

Fig. 12: Smoothed contour in 3D

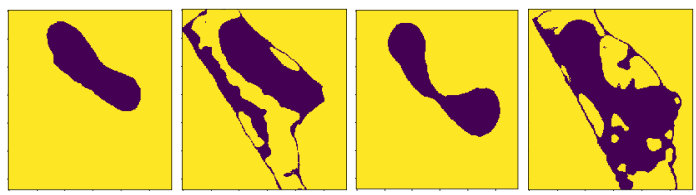

Fig. 13: mitochondria mask generated from running the flow vs. the ground truth segmentation at depth 50 and depth 90

D. Computational time

Speeding up the computation was one of the biggest issues we encountered when implementing the model in 3D. As discussed in [8], the bottleneck regarding run time is associated with the cost of memory. Running Michailovich flow on a (d, m, n)-sized stack yields a memory use of $O(c \cdot m \cdot n \cdot n_s)$, where n_s represents the number of samples in Z space.

In Gaspard's implementation, Numpy's broadcasting was already employed. Compared to computing the gradient pixel-wise, it was observed that the operations sped up by a factor of 10^3. To make most use of Python's c-implemented features, we also switched to running the compiled python code. By running byte-code and saving time by skipping compilation step, the program is now around 3 times faster.

Now we will focus on how the computation is affected by:

1) Contour initialization: As was mentioned, random dense initialization requires more time per iteration and takes longer to form a promising boundary. This makes sense since more and smaller circles need to be evolved in this case. For instance, running the Michailovich flow on a $(50, 200, 200)$ sized region takes around *370 s/iter* with dense initialization compared to *120 s/iter* with local-max initialization.

2) Δ and η: Computation of the derivatives and density estimations accounted for a large portion of the running time. The number of pixels for which derivatives are computed is closely related to Δ, while the narrow band size that determines where density is estimated is defined by η. Therefore, these parameters were carefully tuned such that the algorithm is most efficient while the result is neither randomized nor noisy.

V. Conclusion and future work

In this paper, we adapted a two-dimensional two-channels Michailovich flow implementation in [8] to solve the three-dimensional single-feature mitochondria detection in astrocyte. The model, by design, is based solely on intensity variation and do not require a pre-processing step for smoothing. Following the general classification mechanism, the contour is driven so that Bhattacharyya distance between kernel based estimates of the probability densities of segmentation classes is maximized. Also, With the non-parametric estimates of probability densities, the method can be employed when we do not have much information about the distributions of image features in different segmentation classes [4].

We can conclude from the results that interior contours on source data can be semi-automatically detected with noises, but more organelles than expected were picked up along with mitochondria, which should be expected since we were only using intensity information. Further work can be carried out to solve this problem, for example, by constraining length of the active contour. As a possible extension, the model could also be generalized to have a wider range of applications. As an extension, we can possibly segment smaller objects such as ER and glycogen.

VI. Acknowledgments

I sincerely appreciate my supervisor, Professor Kaleem Siddiqi, for his persistent guidance and valuable insights. He is so nice and patient that it was truly a pleasure to complete this project with his help. I would also like to thank Gaspard Beugnot and Benjamin Kacerovsky for their constructive suggestions.

References

[1] S. Osher and N. Paragios, *Geometric Level Set Methods in Imaging,Vision,and Graphics*. Berlin, Heidelberg: Springer-Verlag, 2003.

[2] J. A. Sethian, "Level set methods : evolving interfaces in geometry, fluid mechanics, computer vision, and materials science," 1996.

[3] [Online]. Available: https://a.carapetis.com/csf/

[4] O. V. Michailovich, Y. Rathi, and A. R. Tannenbaum, "Image segmentation using active contours driven by the bhattacharyya gradient flow," *IEEE Transactions on Image Processing*, vol. 16, pp. 2787–2801, 2007.

[5] S. J. Miller, "Astrocyte heterogeneity in the adult central nervous system," *Frontiers in Cellular Neuroscience*, vol. 12, p. 401, 2018.

[6] J. Kacerovsky, "Structural dissection of cortical astrocytes, their mitochondria, and the endfoot subcompartment," 2019, unpublished thesis.

[7] T. F. Chan and L. A. Vese, "Active contours without edges," *IEEE Transactions on Image Processing*, vol. 10, no. 2, pp. 266–277, 2001.

[8] G. Beugnot, "Segmenting multichannel images: level-sets and variants," 2019, unpublished thesis.

[9] S. Osher and R. Fedkiw, *The Level Set Methods and Dynamic Implicit Surfaces*, 01 2004, vol. 57, pp. xiv+273.